K. J. Somaiya School of Engineering, Mumbai-77

Department of Computer Engineering

LAB CA

Course Name:	**Reinforcement Learning**	**Semester:**	**VII**
Date of Performance:	___ / ___ / _____	**DIV/ Batch No:**	
Student Name:		**Roll No:**	

Reinforcement Learning Lab CA – Mini Project Report

1. Title: A Comparative Study of How Hyperparameters Affect PPO and SAC Agents for Autonomous Lunar Landing

2. Problem Definition & RL Formulation

2.1 Problem Statement

A lander must touch down gently on a landing pad by firing a main engine and two side engines. Every landing happens on a different planet, whose gravity, wind and turbulence are unknown in advance and hidden from the agent, so one control policy has to work on all of them. The task uses the LunarLanderContinuous-v3 environment from OpenAI Gym, now maintained in Gymnasium, a two-dimensional model of the powered descent of a planetary landing.

The project compares two deep reinforcement learning (RL) algorithms, Proximal Policy Optimization (PPO) [1] and Soft Actor-Critic (SAC) [2], to answer two questions: **which setting matters most** for landing (the algorithm, learning rate, network size or training length), and **where performance drops off**, that is, when more training or a higher learning rate makes an agent worse. Training on random planets is domain randomization, and testing on low-gravity planets that training never produces measures out-of-distribution generalization.

2.2 Motivation

Real-world relevance. Powered descent is the riskiest phase of a planetary landing: India's Chandrayaan-2 lander was lost in its final descent in September 2019, while Chandrayaan-3 soft-landed near the lunar south pole on 23 August 2023. Gravity also differs between bodies: the Moon has about 0.17 of Earth's gravity and Mars about 0.38. The simulator's default gravity of −10 is Earth-like, and the low-gravity test planets (−2 to −4) have 0.2–0.4 of it, between the Moon and Mars, so the Moon-like planets are exactly the ones training never sees.

Why reinforcement learning is suitable. There is no dataset of correct thrust commands, but a simulator allows thousands of safe practice landings. The decisive reward comes at the

end of a landing (+100 for coming to rest, −100 for a crash): the credit-assignment problem RL is built for. Classical powered-descent guidance needs an accurate model of the vehicle and the planet, whereas RL learns from interaction, and one policy can cover a whole range of planets.

Why compare hyperparameters. Deep RL is sensitive to hyperparameters, random seeds and implementation details [3], and one SAC run in this study costs 4–10 CPU-hours, so knowing which settings matter, and which are safe, saves compute.

2.3 Objectives

1. Formulate the landing task as a Markov decision process and train PPO and SAC agents on randomized planets.
2. Run a controlled sweep of 2 algorithms × 4 learning rates × 3 network sizes × 3 seeds = 72 training runs, each saved at 6 training lengths, giving 432 agents.
3. Test every agent on the same 300 fixed planets, including unseen low gravity, and measure which setting matters most (an analysis of variance) and where performance drops off.
4. Identify and demonstrate the best agent of each algorithm.

2.4 RL Problem Formulation

Table 1 maps the task onto the components of a Markov decision process, and Table 2 lists the planets used for training and testing.

Table 1. RL problem formulation.

RL Component	Description
Environment	LunarLanderContinuous-v3 (Gymnasium 1.2.3, Box2D physics, 50 steps per second), wrapped so that each landing gets a new random planet (Table 2).
State (S)	8 numbers: x and y position relative to the pad, x and y velocity, angle, angular velocity, and left and right leg contact. Gravity and wind are not included, so the task is partially observed.
Action (A)	2 numbers in [−1, 1]. Main engine: off at 0 or below, 50–100% throttle above 0. Side engines: off while $\|a\| < 0.5$; the sign picks the left or right engine, and the throttle rises from 50% to 100%.
Reward (R)	Each step pays the change in a shaping term: −100 × distance to the pad, −100 × speed, −100 × tilt, +10 per leg on the ground. Firing costs 0.3 × the main engine's throttle and 0.03 × the side engine's throttle per step. The landing ends with +100 for coming to rest, or −100 for a crash or leaving the screen. 200 or more counts as solved.

RL Component	Description
Policy (π)	PPO: a Gaussian whose mean comes from a tanh network and whose log standard deviation is learned; sampled actions are clipped to [−1, 1]. SAC: a tanh-squashed Gaussian from a ReLU network. Tests use the mean action (deterministic).
Discount Factor (γ)	0.99
Episode/Termination Condition	The lander comes to rest, crashes (its body touches the ground), leaves the screen, or reaches 1000 steps (20 s).

Each training landing draws a new planet from bell curves centred on the default planet (a draw outside the test ranges is drawn again), from a generator seeded with the run's seed. The 300 test planets are fixed by a seed (2026): 50 per condition, drawn uniformly within the condition's ranges.

Table 2. Training and test planets (in the simulator's units; default gravity −10).

Planets	Gravity	Wind power	Turbulence	Seen in training?
Training (each landing)	N(−10, 1), kept in −11.9 to −2	|N(0, 7)|, at most 20	|N(0, 0.7)|, at most 2	–
Test: ideal	−10	0	0	yes
Test: moderate wind	−10	5–10	0.5–1	yes
Test: heavy wind	−10	15–20	1.5–2	rarely (3% of landings)
Test: heavy gravity	−11 to −11.9	0	0	yes (14% of landings)
Test: low gravity	−2 to −4	0	0	**no**
Test: random mix	−2 to −11.9	0–20	0–2	partly

2.5 Selected RL Algorithm

- **Name of algorithm:** Proximal Policy Optimization (PPO) [1] and Soft Actor-Critic (SAC) [2], as implemented in Stable-Baselines3.
- **Reason for selecting the algorithm:** the actions are continuous, which rules out DQN and ε-greedy exploration. PPO is the robust on-policy standard and SAC a sample-efficient off-policy method that maximizes entropy: the two main families of deep RL for continuous control.
- **Brief description of the algorithm:** PPO collects a batch of experience with the current policy, then takes several gradient steps on the clipped surrogate objective

$$L^{CLIP}(\theta) = E_t[\, min(\, r_t(\theta)\, \hat{A}_t,\ \ clip(r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon)\, \hat{A}_t\,)\,], \qquad r_t(\theta) = \pi_\theta(a_t \mid s_t) \,/\, \pi_{old}(a_t \mid s_t)$$

where $\hat{A}_t$ is the advantage. The clip ($\varepsilon = 0.2$, a clip range rather than an exploration rate) keeps each update close to the policy that collected the data. SAC maximizes the reward plus the entropy of the policy,

$$J(\pi) = \Sigma_t\, E[\, r_t + \alpha\, H(\pi(\,\cdot \mid s_t))\,]$$

learning two Q-networks (critics) and using the smaller estimate to curb overestimation, storing every transition in a replay buffer, and tuning the temperature α automatically. Section 3.5 gives the update steps of both algorithms.

3. Implementation & Working of RL Agent

3.1 System Architecture / Workflow

Figure 1 shows the learning loop. Before each training landing, the planet wrapper sets a new gravity, wind and turbulence in the simulator. At every step (1/50 s) the agent reads the state s_t and outputs the engine commands a_t, the simulator returns the reward r_t and the next state s_{t+1}, and the learning step updates the policy and its critic: PPO every 2048 steps, and SAC after every step from a replay buffer.

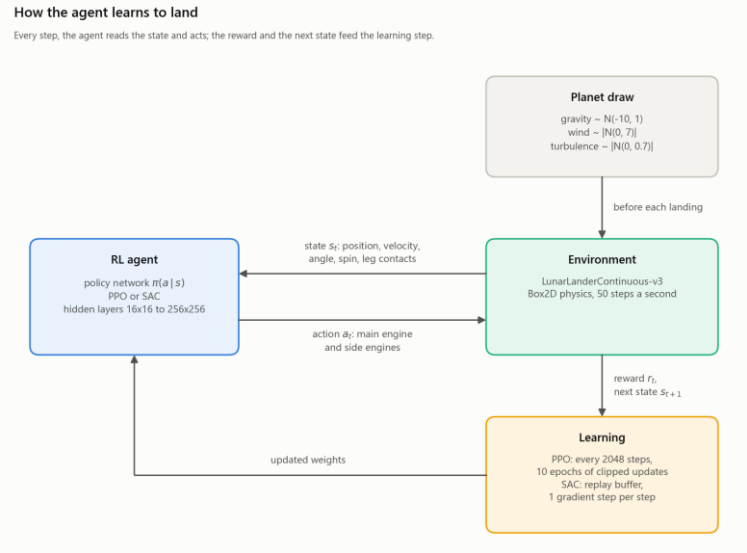


Figure 1. The learning loop: environment → state → RL agent → action → reward → state update → learning, with a new planet before each training landing.

3.2 Tools and Technologies

- **Programming language:** Python 3.13.5.
- **Libraries/frameworks:** Stable-Baselines3 2.8.0 for PPO and SAC, PyTorch 2.11.0, Gymnasium 1.2.3 with Box2D, NumPy 2.4, pandas 3.0.2 and matplotlib 3.10.8.
- **Development environment:** Windows 11, VS Code and Git, on a laptop with an Intel Core i5-12450H (8 cores, 12 threads), 16 GB of RAM and an NVIDIA RTX 3050; training ran on the CPU (Section 3.5).
- **Dataset/simulator:** no dataset; all experience comes from the LunarLanderContinuous-v3 simulator, wrapped to randomize the planet.

3.3 Algorithm Implementation

PPO and SAC are Stable-Baselines3's implementations, with default hyperparameters apart from the learning rate and network size, so both are compared as they are commonly used. This project implements the experiment around them (Figure 2). Three commands run it, and the key excerpts are in Appendix D:

1. `python driver.py train` starts the 72 runs, 11 at a time with one CPU thread each. Each run calls `trainer.train()`, which wraps the simulator in `GaussPlanetWrapper` (a new planet before each landing, from a generator seeded by the run's seed) and builds the model; its `StageSaver` callback logs every landing and saves the weights at the six stages, so an interrupted run resumes from its last stage.
2. `python driver.py test --log` tests each of the 432 agents with `testing.test()`, whose `land()` flies all 300 test planets side by side, with one batched, deterministic network call per step. A landing is *landed* if it ends with +100 for coming to rest, *crashed* if it ends with −100, and *timed out* after 1000 steps. The landings and a summary per agent go to `results/`.
3. `python driver.py plots` calls `plots.make_plots()`, which draws the charts, runs the analysis of variance (`variance_shares()`; Section 4.4, F3) and finds where each setting peaks.

`names.py` holds the agents' names and file paths, and `python diagram.py` draws Figures 1 and 2 from the same settings.

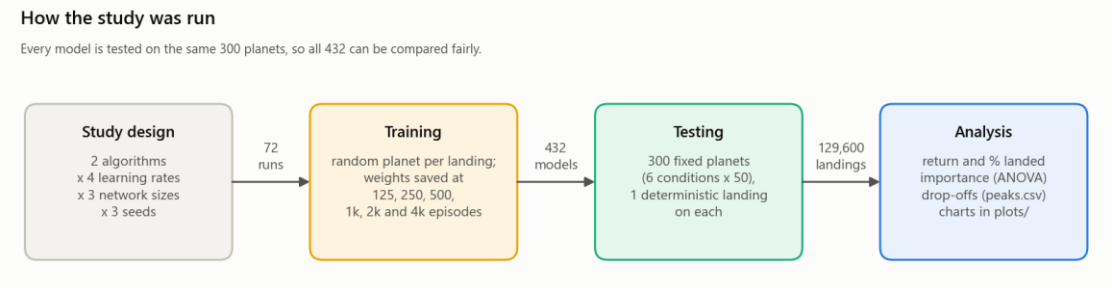


Figure 2. The study pipeline: 72 training runs give 432 saved agents, which make 129,600 test landings for the analysis.

3.4 Hyperparameters / Parameter Settings

Only the learning rate and the network size were varied; everything else is a Stable-Baselines3 default (Table 3). SAC 1 and PPO 1 denote the best setting of each algorithm (Table 5).

Table 3. Hyperparameters.

Parameter	Value
Learning Rate (α)	1e-4, 3e-4, 1e-3 and 3e-3, constant during training; all four with both algorithms (3e-4 is the default).
Discount Factor (γ)	0.99
Exploration Rate (ε)	Not applicable: there is no ε-greedy with continuous actions. PPO samples its Gaussian policy, whose learned standard deviation falls from 1.0 to 0.12–0.33 by 4000 episodes (PPO 1, seed 1). SAC's entropy temperature (SAC's α), tuned towards a target entropy of −2, falls from 1.0 to 0.17 after 125 episodes and 0.04 after 4000 (SAC 1, seed 0). Both decay like ε.
Number of Episodes	4000 training episodes (landings) per run; weights saved after 125, 250, 500, 1000, 2000 and 4000.
Other parameters	Networks: 2 hidden layers of 16, 64 or 256 units, for the actor and the critic separately; seeds 0, 1, 2. PPO: 2048 steps per update, minibatch 64, 10 epochs, GAE $\lambda = 0.95$, clip range 0.2, value-loss weight 0.5, no entropy bonus, gradient norm capped at 0.5. SAC: replay buffer of 1M transitions, batch 256, $\tau = 0.005$, one gradient step per step after 100 random warm-up steps, two critics.

3.5 Training Procedure

Each run trains for 4000 landings on randomized planets and saves its weights at the six stages. **How the networks are updated:** PPO (on-policy) updates every 2048 steps:

1. it computes advantages with GAE ($\gamma = 0.99$, $\lambda = 0.95$) and normalizes them;
2. it takes 10 epochs of 32 minibatches of 64, which is 320 Adam steps, on the clipped surrogate loss plus 0.5 × the value function's squared error (no entropy bonus, gradient norm capped at 0.5);
3. it discards the batch, so the next one comes from the updated policy.

SAC (off-policy) updates after every step:

1. it stores the transition in a replay buffer of 1M transitions and samples a batch of 256;
2. it moves both critics towards $y = r + \gamma(1 - d)(\min_i \bar{Q}_i(s', a') - \alpha \log \pi(a' \mid s'))$, where d marks the end of a landing, a' is drawn from the current policy and $\bar{Q}_i$ are the target critics;
3. it updates the actor to minimize $\alpha \log \pi(a \mid s) - \min_i Q_i(s, a)$, and α to keep the entropy near −2;
4. it moves the target critics: $\bar{Q} \leftarrow \tau Q + (1 - \tau)\bar{Q}$, with $\tau = 0.005$.

Training cost. The 72 runs trained on one laptop, 11 at a time with one CPU thread each, in about 28 hours on 12–13 October 2026 (271 run-hours). A PPO run took 0.7 hours on average (1.3–1.8 M environment steps) and a SAC run 4.7–9.1 hours depending on its size, 7–13 times as long, because SAC updates after every step. SAC 16x16 needed the most steps (2.4 M), as 38% of its training landings hover until the time limit. Testing all 432 agents (129,600 landings) took about 10 minutes.

Engineering challenges.

- **Interrupted training.** A Windows Update restart killed the first attempt after two hours, so runs now save at every stage, write files atomically (a temporary file, then a rename) and resume exactly. The planet generator's state is saved before each draw, because Stable-Baselines3 resets the environment, drawing the next planet, before the callback sees the landing end.
- **CPU, not GPU.** For networks this small the RTX 3050 was no faster than one CPU thread, since each kernel launch costs 30–80 µs under Windows, so training used 11 single-threaded CPU workers.
- **Testing speed.** Flying all 300 landers at once with batched network calls is 3.4 times faster than one at a time.

3.6 Working Demonstration

Output 1 is the console output of testing the best single agent on all 300 planets (97% landed overall, 94% in heavy wind, 90% in low gravity), and Figure 3 shows individual landings. The best SAC agent lands in heavy wind in 3 s (a) and, after a slow, hovering descent, in low gravity, which it never trained on (b). In (c) and (d), one PPO run lands cleanly after 2000 episodes but after 4000 flies off the screen within 1.5 s, a crash: the drop-off of Section 4.4. Each landing matches the logged result for its planet.

Output 1. Console output of `testing.py` for the best single agent (SAC 1, seed 0); "solved" counts landings scoring 200 or more.

```
> python testing.py resources/sac/sac_lr3e-3_net256x256_seed0_ep4000.zip
sac_lr3e-3_net256x256_seed0_ep4000  (trained 4000 episodes, 825,532 steps)
  condition       landings  mean return  solved  landed  crashed  timeout
  ideal                 50        282.6    100%    100%       0%       0%
  moderate_wind         50        288.8    100%    100%       0%       0%
  heavy_wind            50        272.3     94%     94%       4%       2%
  heavy_gravity         50        282.7    100%    100%       0%       0%
  low_gravity           50        259.1     90%     90%       6%       4%
  random_mix            50        273.8     98%     98%       2%       0%
  all                  300        276.6     97%     97%       2%       1%
```

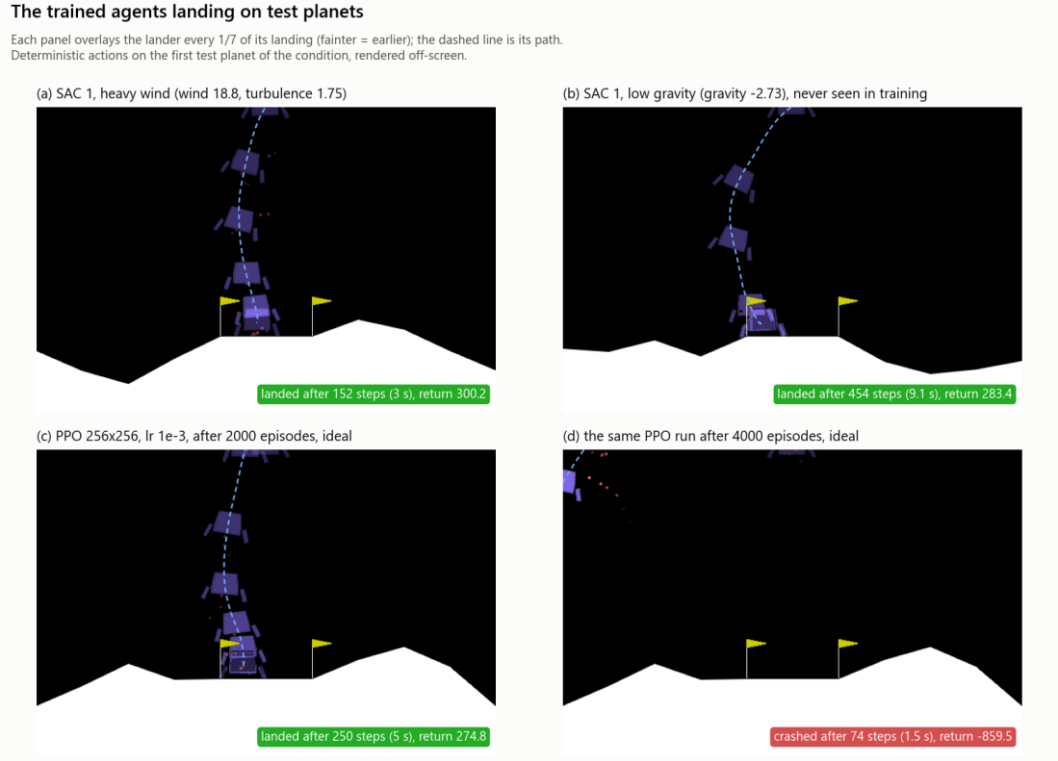


Figure 3. Landings on the first test planet of a condition, with the lander drawn every 1/7 of its landing and its path dashed. SAC 1 is SAC, 256x256, lr 3e-3 (seed 0 here). The same landings play in a window with these commands, in panel order:

```
python testing.py resources/sac/sac_lr3e-3_net256x256_seed0_ep4000.zip --render --conditions heavy_wind
python testing.py resources/sac/sac_lr3e-3_net256x256_seed0_ep4000.zip --render --conditions low_gravity
python testing.py resources/ppo/ppo_lr1e-3_net256x256_seed1_ep2000.zip --render --conditions ideal
python testing.py resources/ppo/ppo_lr1e-3_net256x256_seed1_ep4000.zip --render --conditions ideal
```

4. Results, Analysis & Viva

4.1 Experimental Results

All 432 agents were tested on the same 300 planets, one deterministic landing each. A *setting* is one algorithm, learning rate and network size, and its result is the mean of its 3 seeds. Table 4 gives every setting after 4000 episodes, and Table 5 the three best of each algorithm.

Table 4. Mean test return (landed %) of every setting after 4000 training episodes, averaged over 3 seeds. Bold: the best setting of each algorithm.

Learning rate	PPO			SAC		
	16x16	**64x64**	**256x256**	**16x16**	**64x64**	**256x256**
1e-4	121 (67%)	209 (79%)	**220 (85%)**	201 (80%)	234 (88%)	265 (95%)
3e-4	113 (66%)	175 (73%)	212 (86%)	135 (59%)	246 (90%)	256 (94%)
1e-3	161 (71%)	83 (48%)	−355 (4%)	78 (43%)	257 (91%)	271 (96%)
3e-3	149 (63%)	−187 (18%)	−968 (0%)	149 (67%)	244 (89%)	**275 (96%)**

Table 5. The three best settings of each algorithm, all at their best after 4000 episodes, and SAC 4, at its best after 2000. Seed range: worst to best seed. ≈: estimated from the share of each run's steps done by 2000 episodes.

Rank	Setting	Mean return	Seed range	Landed %	Steps per run	Hours per run
SAC 1	SAC, 256x256, lr 3e-3	275.1	273.8–276.6	96.2	0.83–0.92 M	8.1–9.8
SAC 2	SAC, 256x256, lr 1e-3	271.4	262.7–277.9	96.0	0.90–0.94 M	8.8–9.1
SAC 3	SAC, 256x256, lr 1e-4	264.8	258.0–270.3	95.3	1.10–1.24 M	9.1–10.1
SAC 4	SAC, 64x64, lr 3e-3 (2000 episodes)	264.3	255.1–272.8	93.0	0.62–0.77 M	≈2.0–2.6
PPO 1	PPO, 256x256, lr 1e-4	220.4	214.9–223.7	85.0	1.71–1.88 M	0.8–1.0
PPO 2	PPO, 256x256, lr 3e-4	212.3	197.0–234.2	85.5	1.53–1.56 M	0.8–0.9
PPO 3	PPO, 64x64, lr 1e-4	209.3	191.6–229.5	79.5	1.76–1.91 M	0.8–0.9

The top three of each algorithm are at their best at 4000 episodes, and ranking by landed % picks the same settings (only PPO 1 and 2 swap). SAC 4 (64x64, lr 3e-3) peaks after 2000 episodes at 264.3, level with SAC 3 (264.8) in about a quarter of the training time (2.0–2.6 hours against 9.1–10.1). The winners' seed ranges do not overlap (SAC 1: 273.8–276.6; PPO 1: 214.9–223.7), so the gap is not seed luck.

4.2 Performance Metrics

- **Cumulative reward (return):** the total reward of one landing; 200 or more counts as solved (the dashed line in the charts).
- **Average reward (mean return):** averaged over an agent's 300 test landings (or a condition's 50), then over a setting's 3 seeds; the seed range (worst to best seed) shows the role of luck, and the standard deviation (SD) per landing the consistency.
- **Success rate (landed %):** the share of landings that came to rest; the others crashed (hit the ground or left the screen) or timed out (1000 steps). It is reported with the mean return, as the two rank settings differently.
- **Number of steps:** steps per landing (50 per second); long landings mean hovering.
- **Convergence:** the mean test return changes by less than 10 points between 2000 and 4000 episodes.
- **Accuracy/performance improvement:** from 125 to 4000 episodes, SAC 1 rises from −113 (25% landed) to 275 (96%), and PPO 1 from −201 (0%) to 220 (85%).
- **Importance (η^2):** a setting's share of the spread between agents (Section 4.4, F3).

4.3 Graphs and Visualization

Figures 4 and 6 are the template's reward and success-rate graphs, and Figure 5 adds the test return; steps per episode are summarized in Section 4.4. In each, a panel is one algorithm and network size and each line one learning rate (mean of 3 seeds; band: worst to best seed), on a logarithmic episode axis.

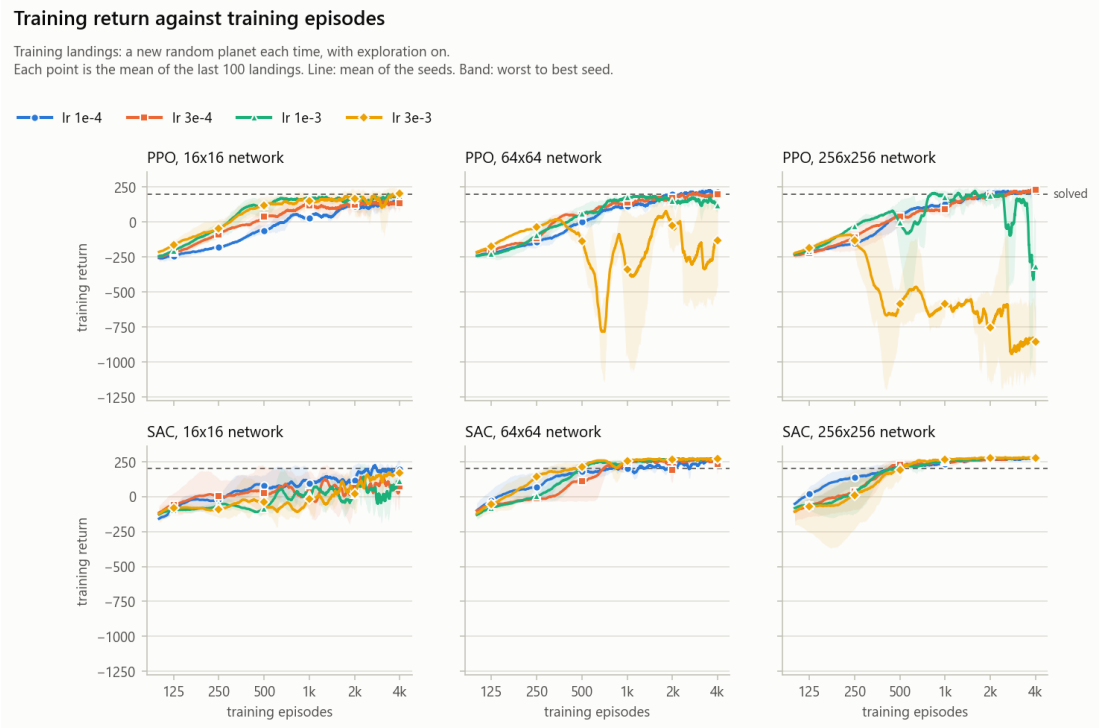


Figure 4. Reward vs. episode in training: mean return of the last 100 training landings, each on a new random planet with exploration on. The PPO collapses appear in training too.

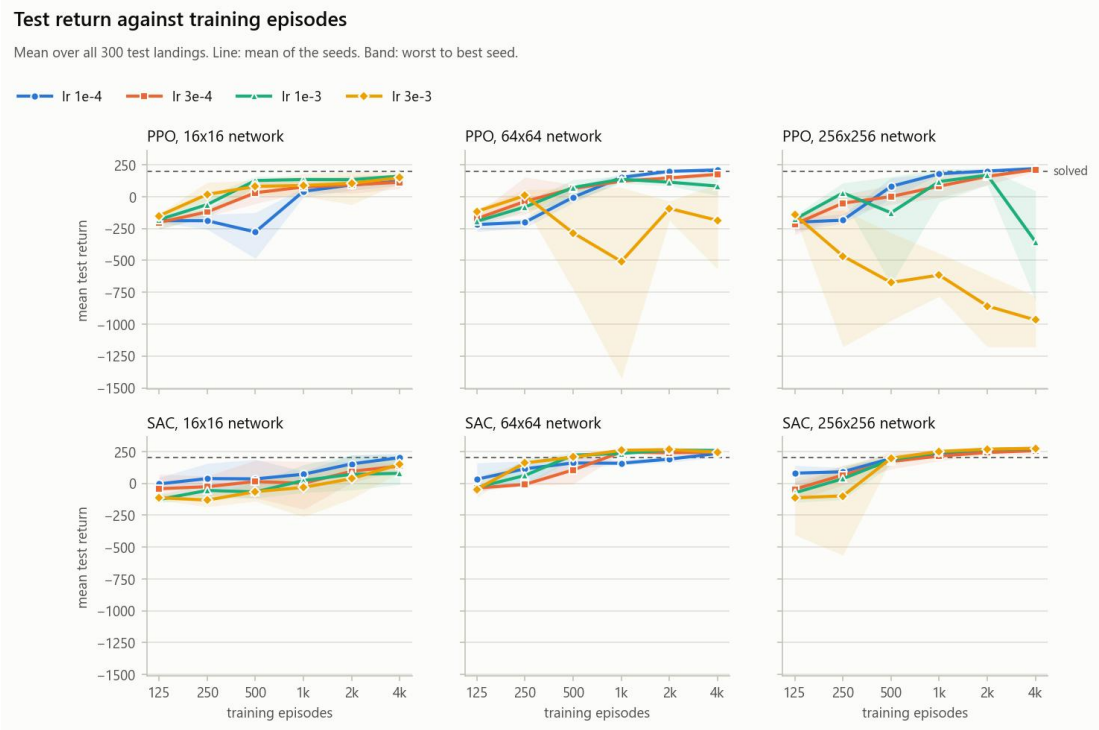


Figure 5. Mean test return vs. training episodes (300 test landings): where performance drops off. The y axis is stretched by the PPO collapses, down to −968.

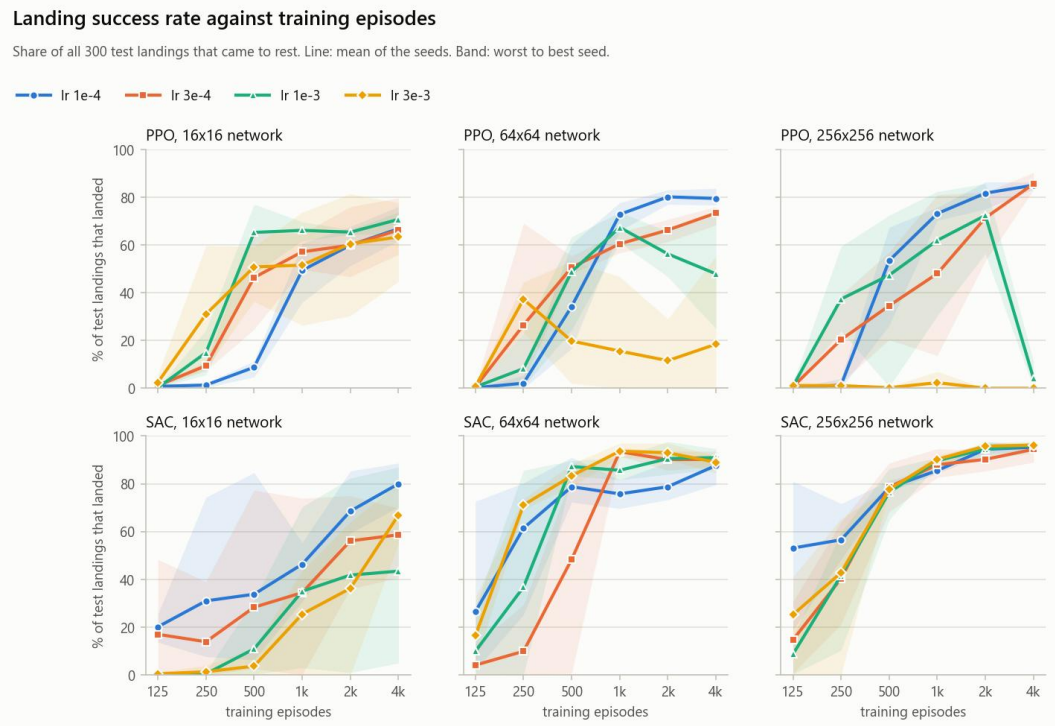


Figure 6. Success rate vs. episode: the share of the 300 test landings that came to rest.

4.4 Result Analysis

What did the agent learn?

SAC 1 learned to land on 96% of the test planets: 100% in ideal conditions, 98% in heavy wind and 85% on unseen low gravity (Table 7), with a mean return of 275. The steps per training landing show the behaviour developing. PPO first crashes within 100–140 steps (around 125 episodes), then hovers to avoid the −100 crash (650–880 steps between 250 and 1000 episodes), and finally lands in 300–475 steps (Figure A1 shows one run in each phase). SAC 64x64 and 256x256 hover from the start (300–900 steps) and shorten their landings to 190–330 steps; SAC 16x16 never stops hovering (37% of its last 500 training landings time out). Exploration shrank as they learned: PPO 1's action noise from 1.0 to 0.12–0.33, and SAC 1's temperature from 1.0 to 0.04.

How did performance change during training?

SAC 256x256 improves at every stage at every learning rate (Figure 5); one seed at lr 3e-3 fell to −563 at 250 episodes but recovered to 185 by 500 (the wide band). PPO with lr 1e-3 or more on 64x64 or 256x256 collapses, in training too (Figure 4): 64x64 at lr 3e-3 peaks at 250 episodes and ends at −187; 64x64 at lr 1e-3 peaks at 1000 (136) and ends at 83; 256x256 at lr 1e-3 peaks at 2000 (169) and ends at −355; and 256x256 at lr 3e-3 never learns (−968). All other settings are at their best at 4000 episodes, except two SAC 64x64 settings that dip 4 and 20 points after 2000 (F4).

Did the agent converge?

By the definition in Section 4.2, 5 of the 24 settings converged: four good SAC settings (64x64 at lr 3e-4 and 1e-3, 256x256 at lr 1e-3 and 3e-3; 246–275), and SAC 16x16 at lr 1e-3, which stalled at 78 (seeds 21, −9 and 222). Most other SAC settings were still gaining 14–112 points from 2000 to 4000 episodes, and no PPO setting converged: the stable ones were gaining 11–54 points and the unstable ones falling by 30–524.

What observations can be made from the graphs?

Four findings stand out. Explanations are marked as likely: only the evidence was measured.

F1. SAC beats PPO per episode and per sample (Table 4, Figure 6). The best PPO setting (220, 85% landed) only matches the average SAC setting (218, 82%), and all eight SAC 64x64 and 256x256 settings beat it (88–96% landed, against at most 86% for PPO), with about half the environment steps (SAC 1: 0.83–0.92 M; PPO 1: 1.71–1.88 M). **Fairness:** the budget was equal episodes, not compute. A SAC run took 7–13 times as long, and per estimated CPU-minute PPO is the more efficient early on: 256x256 at lr 1e-4 reaches 179 (73% landed) after 1000 episodes in about 17 minutes, while SAC 64x64 at lr 3e-3 needs about 24 minutes to reach 160. Later the two are about even (34 minutes for that PPO setting to reach 201, 43 for that SAC setting to reach 206). So SAC wins per episode and per sample, not per hour.

F2. PPO has a learning-rate cut-off above 16x16, and can collapse late (Figures 4 and 5).

- A learning rate of 1e-3 or more breaks PPO on every network larger than 16x16 (−968 to 83), while 1e-4 and 3e-4 reach 175–220: a cut-off, not a gradual trend. The 16x16 network tolerates every learning rate (113–161) and learns fastest with a high one (126 after 500 episodes with lr 1e-3, against −277 with lr 1e-4), but never reaches 200.
- The collapse can come late: 256x256 at lr 1e-3 is healthy at 2000 episodes (169, 72% landed) but ends at −355 (4% landed; Figure 3c–d), so a short study would pick it.
- **Likely cause.** With a constant high learning rate, each update makes 320 Adam steps on one batch; Adam moves every weight by about the learning rate per step, and wider layers add up more of these moves. The clip only stops the gradient for samples already outside 0.8–1.2, so it does not enforce a trust region. Once the mean action is far outside [−1, 1], nearly every sampled action is clipped to the same edge, the advantages no longer point the way back, and, being on-policy, a broken policy only collects data from its own broken behaviour.
- **Evidence.** On the states they visit on 60 test planets, the three lr 3e-3 256x256 policies output mean actions of median size 4–51 and up to 58 (the valid range is ±1), so 99–100% of their actions are clipped. Their action noise grew from a standard

deviation of 1.0 to 1.6–3.2, while PPO 1's shrank to 0.12–0.33, and they crash on every planet, after 1.6 s on average.

- **Why SAC stays stable (likely):** its 1M-transition replay buffer, slow target critics, twin critics and entropy bonus damp a bad update, and tanh squashing keeps actions in range.

F3. Which setting matters most depends on the measure (Table 6, Figure A3). A four-way analysis of variance gives each setting's share, η^2, of the total sum of squares over the 432 agents; what is left between the 3 seeds of the same settings is seed luck.

- By mean return, training length (12%) and the algorithm (12%) lead the single settings, but combinations explain 45% and seed luck 23%, mostly because of the unstable PPO corner (lr 1e-3 or 3e-3 on 64x64 or 256x256, 17% of the agents). Its returns near −1000 dominate the squared spread, causing 72% of the seed-luck share and the largest combined bar in Figure A2 (22%).
- By landed %, which is bounded and less affected by collapses, training length matters most (40%). Without the unstable corner it explains 49% of the spread in return and combinations only 15%, while network size rises from 1% to 10.5%: inside the corner a bigger network is worse and elsewhere better, so the two effects cancel out.
- Among the 72 final agents the algorithm matters most (21% of the spread in landed %; learning rate 15%, network size 1%). Per condition (Figure A3), its share grows from 9% in ideal to 17% in heavy wind, and seed luck reaches 29% in low gravity.

Table 6. Share of the spread between agents explained by each setting (η^2, %). The unstable PPO corner is lr 1e-3 or 3e-3 on 64x64 or 256x256 (72 of the 432 agents).

Setting	Mean return	Landed %	Mean return, without the unstable corner
training episodes	12.0	40.4	48.8
algorithm	11.9	7.7	7.4
learning rate	6.8	2.2	0.4
network size	1.3	4.4	10.5
settings combined	45.1	29.0	14.7
seed (luck)	23.0	16.2	18.1

F4. Generalization and failure modes (Table 7, Table A1).

- Heavy wind widens SAC's lead (PPO 1 falls from 93% landed in ideal to 81%, SAC 1 from 100% to 98%) and rewards training most: from 125 to 4000 episodes the average SAC setting gains 298 points there, against 262 in ideal. Heavy gravity, inside the training range, behaves like ideal.
- **Low gravity is the hardest condition:** at 4000 episodes SAC lands 61% there against 80–90% elsewhere, and PPO 42% against 46–68%, and training helps least

there (from 125 to 4000 episodes the average SAC setting gains 201 points and the average PPO setting 117, against 262 and 188 in ideal). Likely, the agent cannot tell which planet it is on (gravity is not in the state and the network has no memory), so thrust learned for Earth-like gravity lifts it too much and it hovers: SAC 1 times out on 10% of low-gravity landings (at most 2.7% elsewhere) and takes 457 steps, not 189–286.

- **PPO crashes, SAC hovers:** PPO 1's failures are nearly all crashes (14.2% crashed, 0.8% timed out); in low gravity the top three SAC settings timed out 48 times and crashed 24 times in 450 landings.
- **More training can over-specialize:** SAC 64x64 at lr 3e-3 loses 20 points from 2000 to 4000 episodes, almost all in low gravity (233 to 164; 79% to 59% landed), while ideal rises from 270 to 274. With one setting and 3 seeds, it is a sign, not proof.
- **SAC 16x16** depends more on its seed than on its learning rate (at lr 1e-3 the seeds end at 21, −9 and 222). Likely, its 16-unit critic is too coarse and hovering avoids the −100 crash, yet a 16x16 policy can land: PPO 16x16 lands 63–71%.

Table 7. Landed % of the six best settings in each test condition (150 landings per condition: 50 planets × 3 seeds).

Condition	SAC 1	SAC 2	SAC 3	PPO 1	PPO 2	PPO 3
ideal	100.0	100.0	100.0	92.7	94.7	86.0
moderate wind	99.3	99.3	100.0	92.7	89.3	86.7
heavy wind	98.0	98.7	98.0	81.3	78.0	69.3
heavy gravity	98.7	99.3	99.3	86.0	85.3	83.3
low gravity	85.3	84.0	82.7	74.0	77.3	69.3
random mix	96.0	94.7	92.0	83.3	88.7	82.0
all 300	**96.2**	**96.0**	**95.3**	**85.0**	**85.5**	**79.5**

How effective is the learned policy?

SAC 1 is reliable inside its training conditions: in ideal, both winds and heavy gravity it lands 98–100% of the time, with returns of 279–286 and an SD per landing of 16–33 (Table A1). Low gravity is its weak spot (85% landed, SD 80), where it degrades but does not fail. PPO 1 is less reliable (85% landed, 81% in heavy wind) but about 10 times cheaper to train.

4.5 Limitations

- Only 3 seeds per setting (seed luck explains 23% of the spread in return), and mean return is sensitive to collapses, which is why landed % is reported alongside it.
- Only the learning rate and network size were varied; the budget was equal episodes, not equal compute; and most settings had not converged by 4000 episodes.
- A two-dimensional simulator with unlimited fuel and its own shaped reward; low gravity is never trained on, and the test set is 300 fixed planets.

- The explanations in F2 and F4 are likely, not proven: PPO's KL divergence and clip fraction were not logged.

4.6 Future Scope

- Train on to 8000 episodes to find where the 18 settings still best at 4000 drop off, with more seeds, and log PPO's KL divergence and clip fraction.
- Try PPO with a learning-rate schedule or a KL target, and tune the other hyperparameters, for example with Optuna.
- Put low gravity into training or use a curriculum, give the agent the planet's properties or a recurrent network, compare TD3, DDPG and DQN, and move towards 6-DOF landing and sim-to-real transfer.

4.7 Conclusion

This project trained PPO and SAC agents to land on randomized planets they cannot observe: 72 runs, 432 saved agents, each tested on the same 300 planets, and an analysis of variance.

SAC won per episode and per sample: its best setting (256x256, lr 3e-3) reached a mean return of 275 and landed 96% of the time, against 220 and 85% for the best PPO setting (256x256, lr 1e-4), with non-overlapping seed ranges. A cheaper setting, SAC 64x64 at lr 3e-3, matched the third-best SAC setting after 2000 episodes in about a quarter of the training time. What matters most depends on the measure; without the unstable PPO corner, training length explains about half of the spread. PPO needs a learning rate of 3e-4 or lower above 16x16 and can collapse late, while SAC is insensitive to the learning rate at 64x64 and 256x256. Low gravity is the hardest condition, and as a PPO run is 7–13 times faster, the two are roughly even per CPU-minute.

5. References

1. J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
2. T. Haarnoja, A. Zhou, P. Abbeel and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. 35th Int. Conf. Machine Learning (ICML)*, PMLR 80, 2018, pp. 1861–1870.
3. P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup and D. Meger, “Deep reinforcement learning that matters,” in *Proc. AAAI Conf. Artificial Intelligence*, 2018, pp. 3207–3214.

Appendix

A. Source code / GitHub repository link

The code, test results and charts are at https://github.com/dark756/RL-IA. The trained agents (`resources/`) and the training logs (`logs/`) are not in the repository because of their size. To reproduce the study:

```
# Python 3.13.5 with torch 2.11.0, stable-baselines3 2.8.0, gymnasium[box2d] 1.2.3,
# numpy 2.4, pandas 3.0.2 and matplotlib 3.10.8
python driver.py train --cpu-workers 11 --gpu-workers 0   # 72 runs, about 28 hours
python driver.py test --log                               # 432 agents on 300 planets, ~10 minutes
python driver.py plots                                    # charts, importance shares and peaks
python diagram.py                                         # Figures 1 and 2
# Seeds 0, 1 and 2; the test planets are fixed by TEST_SEED = 2026 in testing.py
```

B. Additional screenshots and charts

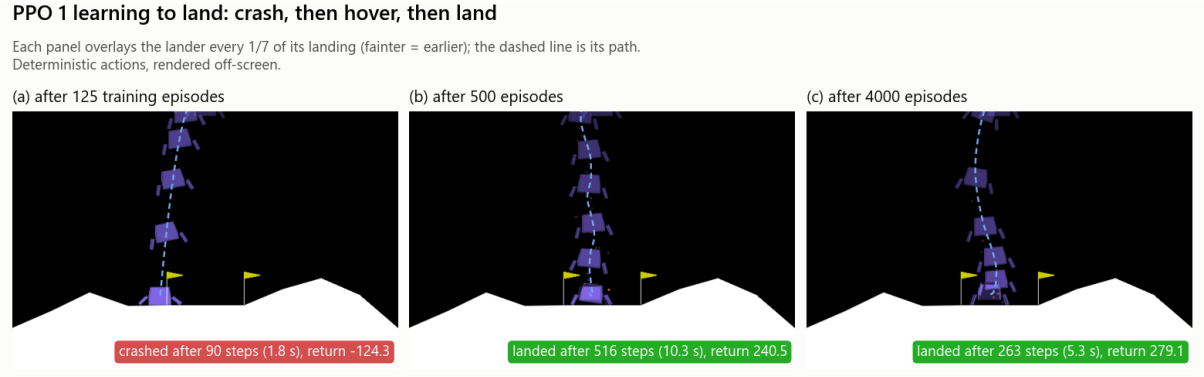


Figure A1. PPO 1 (PPO, 256x256, lr 1e-4, seed 1) on the first ideal test planet after 125, 500 and 4000 training episodes: it crashes, then hovers slowly down, then lands cleanly. Flown alone, the 500-episode landing takes 516 steps, against 495 in the batched test, a small rounding difference. Commands:

```
python testing.py resources/ppo/ppo_lr1e-4_net256x256_seed1_ep125.zip --render --conditions ideal
python testing.py resources/ppo/ppo_lr1e-4_net256x256_seed1_ep500.zip --render --conditions ideal
python testing.py resources/ppo/ppo_lr1e-4_net256x256_seed1_ep4000.zip --render --conditions ideal
```

K. J. Somaiya School of Engineering, Mumbai-77

Department of Computer Engineering

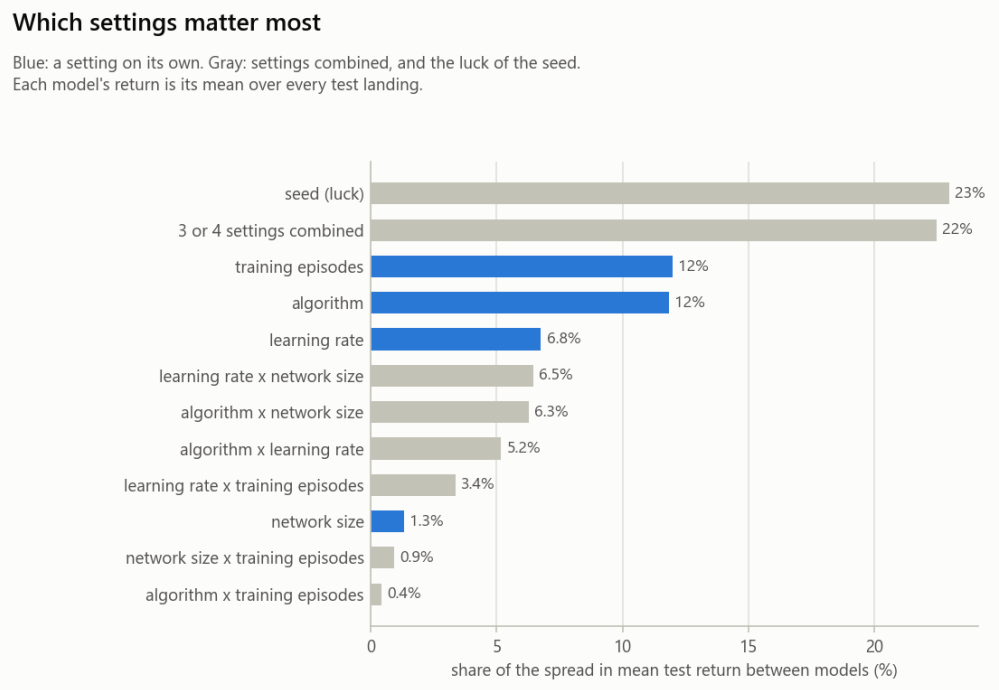


Figure A2. Which settings matter most, with every combination of settings shown separately. The 45% "settings combined" in Table 6 is the sum of the seven gray bars for combinations; the largest, "3 or 4 settings combined" (22%), is the unstable PPO corner.

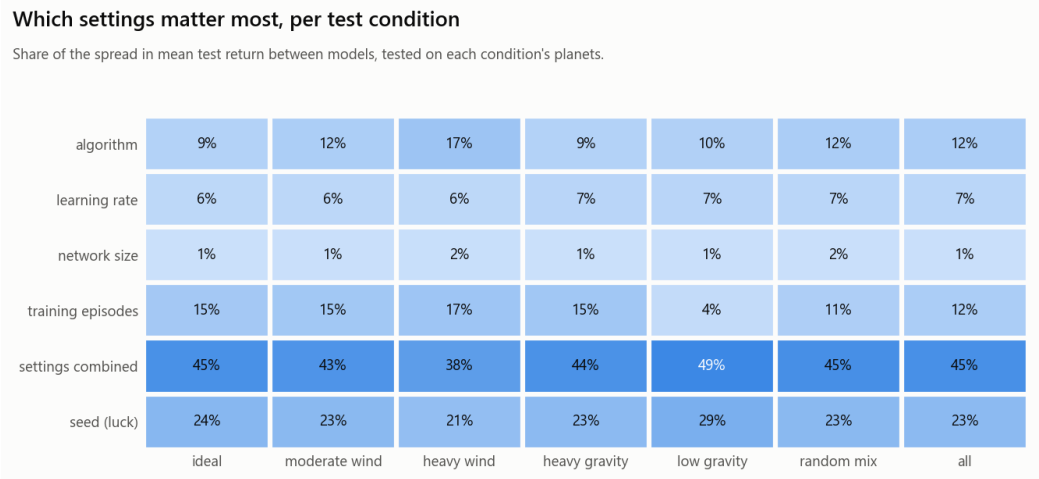


Figure A3. Each setting's share of the spread in mean test return between agents, overall ("all") and per test condition. Compare down a column: each column is a share of its own condition's spread.



Figure A4. How the test landings ended, by algorithm, condition and training stage. All learning rates, sizes and seeds are pooled, so PPO's bars include the collapsed settings.

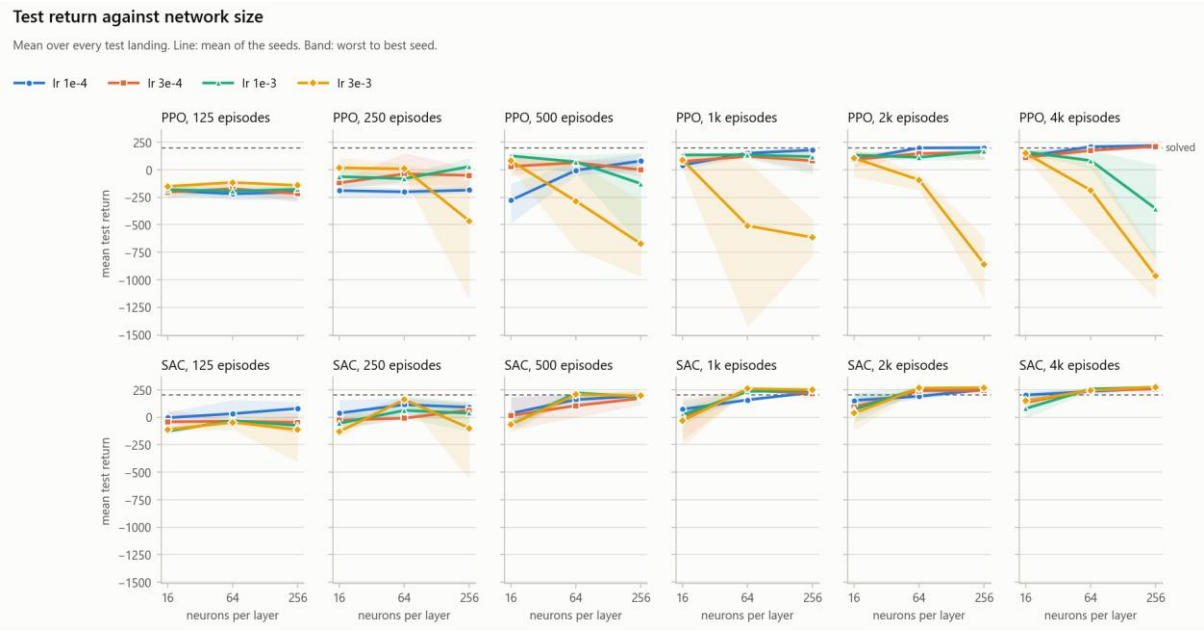


Figure A5. Mean test return vs. network size at each training stage.

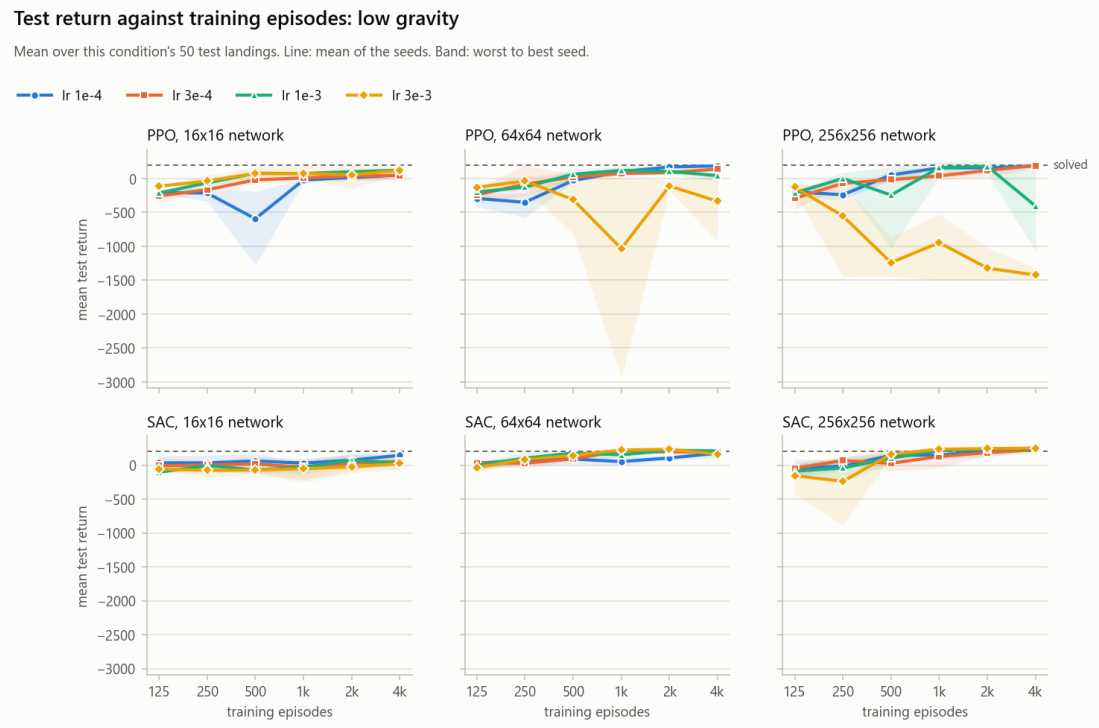


Figure A6. Mean test return vs. training episodes on the low-gravity planets only. The other conditions' charts are in the repository's `plots/conditions/`.

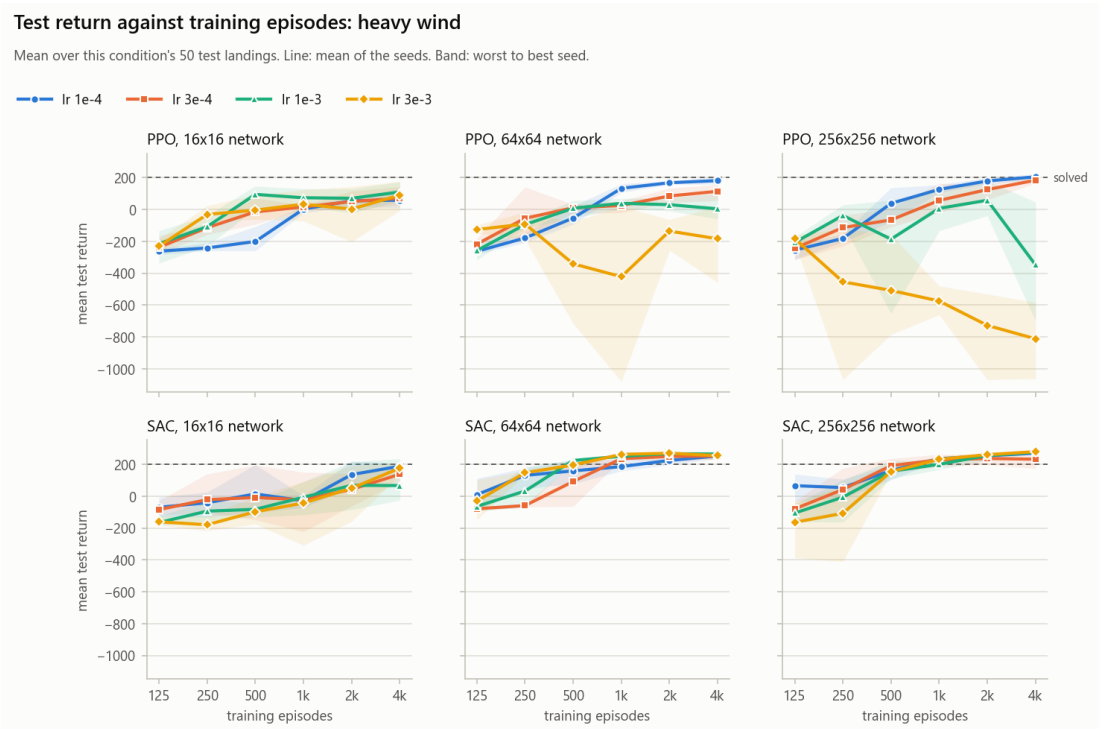


Figure A7. Mean test return vs. training episodes on the heavy-wind planets only.

C. Sample outputs

Output 2. The state and action spaces: `python -c "import gymnasium as gym; e = gym.make('LunarLanderContinuous-v3'); print(e.observation_space); print(e.action_space)"`

```
Box([ -2.5        -2.5       -10.        -10.         -6.2831855 -10.
  -0.         -0.       ], [ 2.5        2.5       10.        10.         6.2831855 10.
  1.         1.       ], (8,), float32)
Box(-1.0, 1.0, (2,), float32)
```

Output 3. The analysis printed by `python driver.py plots`: the importance shares, the ten best settings and stages, and where each setting peaks.

```
What matters most (% of the spread in mean test return between models):
seed (luck)                          23.0
3 or 4 settings combined             22.5
training episodes                    12.0
algorithm                            11.9
learning rate                         6.8
learning rate x network size          6.5
algorithm x network size              6.3
algorithm x learning rate             5.2
learning rate x training episodes     3.4
network size                          1.3
network size x training episodes      0.9
algorithm x training episodes         0.4

Best settings (mean test return over all test landings; mean, worst and best seed):
algorithm   lr    size  train_episodes  mean   min   max
      sac 3e-3 256x256            4000 275.1 273.8 276.6
      sac 1e-3 256x256            4000 271.4 262.7 277.9
      sac 3e-3 256x256            2000 267.8 265.1 272.4
      sac 1e-3 256x256            2000 265.5 253.1 275.7
      sac 1e-4 256x256            4000 264.8 258.0 270.3
      sac 3e-3   64x64            2000 264.3 255.1 272.8
      sac 1e-3   64x64            2000 261.3 250.3 275.2
      sac 3e-3   64x64            1000 258.9 250.3 266.6
      sac 1e-3   64x64            4000 257.5 236.6 270.8
      sac 3e-4 256x256            4000 256.3 224.1 274.0

Where each setting peaks, and how much it drops by the last stage (saved to results/peaks.csv):
algorithm   lr    size  best_episodes  best_return  last_episodes  last_return  drop_after_best
      ppo 1e-3   16x16           4000        161.0           4000        161.0              0.0
      ppo 1e-4   16x16           4000        121.3           4000        121.3              0.0
      ppo 3e-3   16x16           4000        149.0           4000        149.0              0.0
      ppo 3e-4   16x16           4000        112.5           4000        112.5              0.0
      ppo 1e-3   64x64           1000        136.3           4000         83.1             53.2
      ppo 1e-4   64x64           4000        209.3           4000        209.3              0.0
      ppo 3e-3   64x64            250          8.3           4000       -186.7            195.0
      ppo 3e-4   64x64           4000        175.3           4000        175.3              0.0
      ppo 1e-3 256x256           2000        168.9           4000       -354.6            523.5
      ppo 1e-4 256x256           4000        220.4           4000        220.4              0.0
      ppo 3e-3 256x256            125       -141.8           4000       -967.7            825.9
      ppo 3e-4 256x256           4000        212.3           4000        212.3              0.0
      sac 1e-3   16x16           4000         78.3           4000         78.3              0.0
      sac 1e-4   16x16           4000        201.5           4000        201.5              0.0
      sac 3e-3   16x16           4000        148.8           4000        148.8              0.0
      sac 3e-4   16x16           4000        135.3           4000        135.3              0.0
      sac 1e-3   64x64           2000        261.3           4000        257.5              3.8
      sac 1e-4   64x64           4000        234.4           4000        234.4              0.0
      sac 3e-3   64x64           2000        264.3           4000        244.5             19.9
      sac 3e-4   64x64           4000        245.8           4000        245.8              0.0
      sac 1e-3 256x256           4000        271.4           4000        271.4              0.0
      sac 1e-4 256x256           4000        264.8           4000        264.8              0.0
      sac 3e-3 256x256           4000        275.1           4000        275.1              0.0
      sac 3e-4 256x256           4000        256.3           4000        256.3              0.0
```

D. Important code snippets

Listing 1. `trainer.py`, lines 34–64: the wrapper that draws a new planet before every landing.

```
class GaussPlanetWrapper(gym.Wrapper):
    """Like ChaosPlanetWrapper in lunar_random.py, but the planets come from bell curves and a seeded
    generator. It only sets gravity, wind and turbulence before each landing; nothing pushes the lander."""

    def __init__(self, env, seed):
        super().__init__(env)
        self.rng = np.random.default_rng(seed)
        self.rng_state = self.rng.bit_generator.state

    def reset(self, **kwargs):
        self.rng_state = self.rng.bit_generator.state  # saved at each stage, so a run carries on from this planet
        self.gravity = draw(lambda: self.rng.normal(GRAVITY_MEAN, GRAVITY_SPREAD), GRAVITY_LIMITS)
        self.wind = draw(lambda: abs(self.rng.normal(0.0, WIND_SPREAD)), WIND_LIMITS)
        self.turbulence = draw(lambda: abs(self.rng.normal(0.0, TURBULENCE_SPREAD)), TURBULENCE_LIMITS)
        self.env.unwrapped.gravity = self.gravity
        self.env.unwrapped.wind_power = self.wind
        self.env.unwrapped.turbulence_power = self.turbulence
        return self.env.reset(**kwargs)

    def step(self, action):
        obs, reward, te, tr, info = self.env.step(action)
        if te or tr:  # tells the training log which planet this landing was on
            info["planet"] = (self.gravity, self.wind, self.turbulence)
        return obs, reward, te, tr, info


def draw(sample, limits):
    value = sample()
    while not limits[0] <= value <= limits[1]:
        value = sample()
    return float(value)
```

Listing 2. `trainer.py`, lines 145–148 and 78–94: the model (Stable-Baselines3 defaults apart from the learning rate and network size) and the callback that logs every landing and saves each stage.

```
    else:
        policy_kwargs = dict(net_arch=list(size))
        model = ALGOS[algo]("MlpPolicy", env, learning_rate=lr, policy_kwargs=policy_kwargs, seed=seed,
                            device=device, verbose=0)

    def _on_step(self):
        for info in self.locals["infos"]:
            if "episode" not in info:
                continue
            self.episodes += 1
            ret, length = info["episode"]["r"], info["episode"]["l"]
            gravity, wind, turbulence = info["planet"]
            self.log.writerow([self.episodes, self.num_timesteps, round(ret, 2), length,
                               round(gravity, 3), round(wind, 3), round(turbulence, 3)])
            self.recent.append(ret)
            if self.episodes in self.stages:
                self.model.save(os.path.join(self.folder, f"{self.name}_ep{self.episodes}"))
                self.save_progress(self.episodes)
                print(f"{self.name}  episode {self.episodes:>5}  steps {self.num_timesteps:>9,}  "
                      f"last 100 landings {np.mean(self.recent):7.1f}  {(time.time() - self.start) / 60:6.1f}
     min",
                      flush=True)
        return self.episodes < self.stages[-1]
```

Listing 3. `trainer.py`, lines 156–168: saving progress atomically at each stage.

```
        def save_progress(episodes):
            """What the run needs to carry on from this stage, so a restart only loses the work since it."""
            f.flush()
            if os.path.exists(state_file):
                os.remove(state_file)  # a crash mid-save then means starting over, never a mismatched buffer
            if algo == "sac":
                model.save_replay_buffer(buffer_file + ".tmp")
                os.replace(buffer_file + ".tmp", buffer_file)
            seconds = time.time() - started + (state["seconds"] if state else 0)
            with open(state_file + ".tmp", "w", encoding="utf-8") as g:
                json.dump({"episodes": episodes, "steps": model.num_timesteps, "seconds": round(seconds),
                           "device": device, "rng": env.rng_state}, g, indent=1)
            os.replace(state_file + ".tmp", state_file)
```

Listing 4. `testing.py`, lines 81–108: batched deterministic landings and the outcome rule.

```
def land(model, planets, render=False):
    """Gives the model one deterministic landing on each planet. All landers fly side by side,
    so the model picks every lander's action in one call."""
    envs = [gym.make("LunarLanderContinuous-v3", render_mode="human" if render else None,
                     gravity=p["gravity"], enable_wind=True, wind_power=p["wind"],
                     turbulence_power=p["turbulence"]) for p in planets]
    obs = np.array([env.reset(seed=p["seed"])[0] for env, p in zip(envs, planets)])
    returns, steps, outcomes = np.zeros(len(envs)), np.zeros(len(envs), dtype=int), [""] * len(envs)
    flying = list(range(len(envs)))
    while flying:
        actions, _states = model.predict(obs[flying], deterministic=True)
        still_flying = []
        for i, action in zip(flying, actions):
            obs[i], reward, te, tr, info = envs[i].step(action)
            returns[i] += reward
            steps[i] += 1
            if te:  # the last reward is +100 for coming to rest and -100 for a crash or flying off screen
                outcomes[i] = "landed" if reward > 0 else "crashed"
            elif tr:
                outcomes[i] = "timeout"
            if te or tr:
                envs[i].close()
            else:
                still_flying.append(i)
        flying = still_flying
    return [{"env": p["name"], "condition": p["condition"], "gravity": p["gravity"], "wind": p["wind"],
             "turbulence": p["turbulence"], "return": round(float(r), 2), "steps": int(s), "outcome": o}
            for p, r, s, o in zip(planets, returns, steps, outcomes)]
```

Listing 5. `plots.py`, lines 103–119: the analysis of variance behind Table 6.

```
def variance_shares(models):
    """How much of the spread in mean test return between models each setting explains (ANOVA eta squared).
    Whatever no setting or combination of settings explains is the luck of the seed."""
    y = models["return"]
    grand = y.mean()
    total = ((y - grand) ** 2).sum() or 1.0
    shares, effects = {}, {}
    for factor, label in SETTINGS.items():
        effects[factor] = models.groupby(factor)["return"].transform("mean") - grand
        shares[label] = (effects[factor] ** 2).sum() / total
    for a, b in combinations(SETTINGS, 2):
        pair = models.groupby([a, b])["return"].transform("mean") - grand - effects[a] - effects[b]
        shares[f"{SETTINGS[a]} x {SETTINGS[b]}"] = (pair ** 2).sum() / total
    cells = models.groupby(list(SETTINGS))["return"].transform("mean")
    shares["3 or 4 settings combined"] = max(0.0, ((cells - grand) ** 2).sum() / total - sum(shares.values()))
    shares["seed (luck)"] = ((y - cells) ** 2).sum() / total
    return pd.Series(shares) * 100
```

E. Additional tables

Table A1. SAC 1 (SAC, 256x256, lr 3e-3) after 4000 episodes, by test condition (3 seeds, 150 landings per condition).

Condition	Mean return	Seed range	SD per landing	Landed %	Crashed %	Timeout %	Mean steps
ideal	280.8	278.4–282.6	16.3	100.0	0.0	0.0	193
moderate wind	286.2	282.9–288.8	19.3	99.3	0.0	0.7	192
heavy wind	279.7	272.3–284.5	33.0	98.0	1.3	0.7	213
heavy gravity	279.0	274.8–282.7	22.0	98.7	0.7	0.7	189
low gravity	253.2	247.4–259.1	80.0	85.3	4.7	10.0	457
random mix	272.0	266.8–275.4	53.8	96.0	1.3	2.7	286
all	**275.1**	**273.8–276.6**	**45.9**	**96.2**	**1.3**	**2.5**	**255**

Table A2. Mean test return of the six best settings in each test condition, after 4000 episodes (3 seeds).

Condition	SAC 1	SAC 2	SAC 3	PPO 1	PPO 2	PPO 3
ideal	280.8	280.9	274.1	240.8	235.7	224.2
moderate wind	286.2	285.8	281.4	245.9	226.4	228.9
heavy wind	279.7	276.1	270.4	205.3	184.1	181.6
heavy gravity	279.0	280.1	274.0	230.3	221.1	219.8
low gravity	253.2	241.8	247.7	186.0	190.9	187.2
random mix	272.0	263.6	241.4	214.0	215.3	214.0
all 300	**275.1**	**271.4**	**264.8**	**220.4**	**212.3**	**209.3**

F. Use of AI tools

An AI assistant (Claude, by Anthropic) was used to help write the code in the repository and to draft this report. All results come from the experiments described above and were checked against the logged data in the repository.